The Story of AI

From Perceptrons to Reasoning Machines

A Narrative History of Artificial Intelligence and Language Models

May 2026

Contents

Prologue: The Dream 5

1 The Foundations (1943–2014) 7

2 The Transformer Revolution (2017–2019) 13

3 The Age of Scale (2020–2022) 19

4 The Open Revolution (2023) 23

5 Efficiency and Specialization (2024) 27

6 Reasoning and Agency (2024–2025) 31

Epilogue 35

Prologue: The Dream

In the winter of 1943, as the world burned through its second great war, two scientists at the University of Chicago published a paper that almost no one read. Warren McCulloch, a neurophysiologist with a poet's imagination, and Walter Pitts, a self-taught mathematical prodigy who had run away from home at fifteen, proposed something audacious: that the human brain could be modeled as a network of simple logical units, and that this network could compute any function that was computable.

They called their units "neurons," and they showed that networks of these neurons, wired together with the right connections, could perform the logical operations that underpin all of mathematics. Their paper, *A Logical Calculus of the Ideas Immanent in Nervous Activity* (McCulloch & Pitts, 1943), contained no experiments, no data, no working prototype. It was pure theory—a mathematical proof that thought itself might be mechanizable.

This was the founding myth of artificial intelligence: that the mind is a machine, and that building a mind means building the right machine.

The story that follows spans eight decades. It moves from mathematical abstractions to silicon chips, from a single neuron learning to recognize simple shapes to trillion-parameter models that write poetry and prove theorems. It is a story of ideas—some that worked, many that didn't, and a few that changed everything.

It begins, as all good stories do, with a failure.

Chapter 1

The Foundations (1943–2014)

1.1 The Perceptron: A Neuron Learns to See

In 1957, a Cornell psychologist named Frank Rosenblatt built a machine the size of a refrigerator. It had four hundred photocells arranged in a grid, connected through a nest of wires to motor-driven potentiometers that adjusted the strength of each connection. He called it the **Perceptron**.

Rosenblatt's insight, published in *The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain* (Rosenblatt, 1958), was elegantly simple. A Perceptron takes a set of inputs $\mathbf{x} = (x_1, x_2, \ldots, x_d)$, multiplies each by a learned weight w_i, sums them, and applies a threshold:

$$y = \begin{cases} 1 & \text{if } \sum_{i=1}^{d} w_i x_i + b > 0 \\ 0 & \text{otherwise} \end{cases} \tag{1.1}$$

If the output is wrong, adjust the weights: $w_i \leftarrow w_i + \eta \cdot (y_{\text{true}} - y_{\text{pred}}) \cdot x_i$. This is the **perceptron learning rule**—and it is the ancestor of every neural network training algorithm that followed.

The New York Times reported that the Navy expected the Perceptron would soon "be able to walk, talk, see, write, reproduce itself, and be conscious of its existence." Rosenblatt himself predicted that machines would match human intelligence within a decade.

1.1.1 The Winter That Followed

In 1969, Marvin Minsky and Seymour Papert published *Perceptrons*, a devastating mathematical critique. They proved, rigorously, that a single-layer Perceptron could not solve the XOR problem—could not learn that "A or B, but not both" is different from "A and B." The XOR function is not linearly separable; no single line can divide its truth table.

More damningly, they cast doubt on whether multi-layer networks could be trained at all. Without a method to assign credit to hidden layers, deep networks seemed like a dead end.

The result was the **First AI Winter**. Funding dried up. Research programs were cancelled. The dream of machine intelligence went dormant for a decade.

1.2 Backpropagation: Credit Assignment Solved

The problem Minsky and Papert identified was real, but not insurmountable. The solution had actually been discovered years earlier.

In 1986, David Rumelhart, Geoffrey Hinton, and Ronald Williams published *Learning Representations by Back-Propagating Errors* (Rumelhart et al., 1986). Their algorithm, **backpropagation**, solved the credit assignment problem by computing gradients through the chain rule:

$$\frac{\partial \mathcal{L}}{\partial w_{ij}^{(\ell)}} = \frac{\partial \mathcal{L}}{\partial a_j^{(\ell)}} \cdot \frac{\partial a_j^{(\ell)}}{\partial z_j^{(\ell)}} \cdot \frac{\partial z_j^{(\ell)}}{\partial w_{ij}^{(\ell)}} \tag{1.2}$$

where $z_j^{(\ell)} = \sum_i w_{ij}^{(\ell)} a_i^{(\ell-1)}$ is the pre-activation of neuron j in layer ℓ, and $a_j^{(\ell)} = \sigma(z_j^{(\ell)})$ is its activation. The key insight: the error at layer ℓ can be expressed in terms of the error at layer $\ell + 1$, propagating backward through the network:

$$\delta_j^{(\ell)} = \frac{\partial \mathcal{L}}{\partial z_j^{(\ell)}} = \left(\sum_k w_{jk}^{(\ell+1)} \delta_k^{(\ell+1)} \right) \cdot \sigma'(z_j^{(\ell)}) \tag{1.3}$$

This was the missing piece. Multi-layer networks could now be trained end-to-end. The Second AI Spring had begun.

But there was a catch. Deep networks—networks with many layers—suffered from the **vanishing gradient problem**. As errors propagated backward, the sigmoid derivative $\sigma'(z) = \sigma(z)(1 - \sigma(z))$ multiplied at each layer, shrinking gradients exponentially. Early layers learned almost nothing. For another two decades, deep networks remained stubbornly difficult to train.

1.3 Words as Vectors: The Embedding Revolution

While neural networks struggled with depth, a parallel revolution was brewing in how we represent language.

For most of computing history, words were discrete symbols—indices in a dictionary. The word "king" was token #4837; "queen" was token #9214. There was no mathematical relationship between them. This made language a hostile medium for neural networks, which thrive on continuous spaces.

Then came a series of breakthroughs that changed everything.

1.3.1 Word2Vec (Mikolov et al., 2013)

Tomas Mikolov and his colleagues at Google asked a different question: what if we could learn a vector representation for each word such that words with similar meanings have similar vectors? Their paper, *Efficient Estimation of Word Representations in Vector Space* (Mikolov et al., 2013), introduced **Word2Vec**.

The insight was to use a simple prediction task as the training signal. In the **Skip-gram** variant, given a word w_t, predict its surrounding context words $w_{t-2}, w_{t-1}, w_{t+1}, w_{t+2}$. The model learns embeddings $\mathbf{v}_w \in \mathbb{R}^d$ such that:

$$p(w_{t+j} \mid w_t) = \frac{\exp(\mathbf{v}_{w_{t+j}}^\top \mathbf{v}_{w_t})}{\sum_{w' \in V} \exp(\mathbf{v}_{w'}^\top \mathbf{v}_{w_t})} \tag{1.4}$$

The result was astonishing. The learned vectors captured semantic relationships as linear algebraic operations:

$$\mathbf{v}_{\text{king}} - \mathbf{v}_{\text{man}} + \mathbf{v}_{\text{woman}} \approx \mathbf{v}_{\text{queen}} \tag{1.5}$$

This was not programmed. It *emerged* from the training data. The model had discovered that "king" and "queen" differ primarily along a gender dimension, and that this dimension is consistent across word pairs. For the first time, machines could reason about words mathematically.

1.3.2 GloVe (Pennington et al., 2014)

Jeffrey Pennington and his collaborators at Stanford refined this insight with **GloVe** (*Global Vectors for Word Representation*, 2014). Where Word2Vec used local context windows, GloVe leveraged the global co-occurrence statistics of the entire corpus. The model factors the co-occurrence count matrix $\mathbf{X}$:

$$\mathbf{v}_i^\top \mathbf{v}_j + b_i + \tilde{b}_j = \log X_{ij} \tag{1.6}$$

where X_{ij} is how often word j appears in the context of word i. GloVe produced embeddings that captured both local and global linguistic patterns, and trained faster than Word2Vec.

1.4 The Recurrence Era: RNNs, LSTMs, and GRUs

With words now representable as vectors, the next question was: how do we process sequences of words?

1.4.1 Recurrent Neural Networks

A recurrent neural network (RNN) processes a sequence $\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_T$ by maintaining a hidden state $\mathbf{h}_t$ that serves as memory:

$$\mathbf{h}_t = \tanh(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{xh}\mathbf{x}_t + \mathbf{b}) \tag{1.7}$$

The output at each step is $\hat{\mathbf{y}}_t = \text{softmax}(\mathbf{W}_{hy}\mathbf{h}_t)$. This is elegantly simple—the network "remembers" past inputs through $\mathbf{h}_t$.

But the vanishing gradient problem returned with a vengeance. When unrolled through time, deep RNNs multiply the same weight matrix $\mathbf{W}_{hh}$ over and over. If the largest eigenvalue of $\mathbf{W}_{hh}$ is less than 1, gradients vanish. If greater than 1, they explode. Either way, long-range dependencies remain out of reach.

1.4.2 Long Short-Term Memory (Hochreiter & Schmidhuber, 1997)

Sepp Hochreiter and Jürgen Schmidhuber solved this with one of the most important papers in neural network history: *Long Short-Term Memory* (1997). The LSTM introduces a **memory cell** $\mathbf{c}_t$ that can maintain information over arbitrary time spans, protected by learned gates:

$$\mathbf{f}_t = \sigma(\mathbf{W}_f \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f) \quad \text{(forget gate)} \tag{1.8}$$

$$\mathbf{i}_t = \sigma(\mathbf{W}_i \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i) \quad \text{(input gate)} \tag{1.9}$$

$$\tilde{\mathbf{c}}_t = \tanh(\mathbf{W}_c \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c) \quad \text{(candidate memory)} \tag{1.10}$$

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t \quad \text{(memory update)} \tag{1.11}$$

$$\mathbf{o}_t = \sigma(\mathbf{W}_o \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o) \quad \text{(output gate)} \tag{1.12}$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t) \tag{1.13}$$

The forget gate $\mathbf{f}_t$ can be 1 (remember everything) or 0 (forget everything) or anything in between. The input gate controls what new information enters memory. The output gate controls what the network reveals. This gating mechanism creates **constant error carousels**—pathways through which gradients can flow unchanged across many time steps.

For twenty years, the LSTM was the undisputed king of sequence modeling. It powered speech recognition at Google, machine translation at Facebook, and handwriting recognition at Apple. If you used any AI system between 2000 and 2017, an LSTM was almost certainly doing the heavy lifting.

1.4.3 GRU (Cho et al., 2014)

Kyunghyun Cho and colleagues simplified the LSTM with the **Gated Recurrent Unit** (*Learning Phrase Representations using RNN Encoder-Decoder*, 2014). The GRU merges the cell state and hidden state, and uses only two gates:

$$\mathbf{z}_t = \sigma(\mathbf{W}_z \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t]) \quad \text{(update gate)} \tag{1.14}$$

$$\mathbf{r}_t = \sigma(\mathbf{W}_r \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t]) \quad \text{(reset gate)} \tag{1.15}$$

$$\tilde{\mathbf{h}}_t = \tanh(\mathbf{W} \cdot [\mathbf{r}_t \odot \mathbf{h}_{t-1}, \mathbf{x}_t]) \tag{1.16}$$

$$\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t \tag{1.17}$$

The GRU has fewer parameters than the LSTM and often performed comparably. It became the go-to recurrent unit for many applications.

1.5 Sequence-to-Sequence: The Birth of Neural Translation

In 2014, two papers appeared almost simultaneously, both proposing the same architecture for machine translation. Ilya Sutskever, Oriol Vinyals, and Quoc Le at Google published *Sequence to Sequence Learning with Neural Networks* (Sutskever et al., 2014). Kyunghyun Cho and colleagues published *Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation* (Cho et al., 2014).

Both proposed the same elegant structure: an **encoder** RNN reads the source sentence and compresses it into a fixed-length vector $\mathbf{c}$. A **decoder** RNN generates the target sentence, one word at a time, conditioned on $\mathbf{c}$:

$$\mathbf{h}_t^{\text{enc}} = \text{LSTM}(\mathbf{h}_{t-1}^{\text{enc}}, \mathbf{x}_t) \tag{1.18}$$

$$\mathbf{c} = \mathbf{h}_T^{\text{enc}} \quad \text{(the final hidden state as context)} \tag{1.19}$$

$$\mathbf{h}_t^{\text{dec}} = \text{LSTM}(\mathbf{h}_{t-1}^{\text{dec}}, [\mathbf{y}_{t-1}, \mathbf{c}]) \tag{1.20}$$

$$p(y_t \mid y_{<t}, \mathbf{x}) = \text{softmax}(\mathbf{W}_{\text{out}}\mathbf{h}_t^{\text{dec}}) \tag{1.21}$$

This was a revelation. For the first time, an end-to-end neural network could translate between languages without hand-crafted features, phrase tables, or linguistic rules. It just worked.

But the architecture had a fatal flaw: the **information bottleneck**. The entire source sentence had to be compressed into a single fixed-length vector $\mathbf{c}$. For long sentences, this vector became a bottleneck—crucial information from the beginning of the sentence was lost by the time the decoder needed it.

1.6 Attention: The Missing Piece (Bahdanau et al., 2015)

Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio published a paper that would change everything: *Neural Machine Translation by Jointly Learning to Align and Translate* (Bahdanau et al., 2015).

Their insight was profound: instead of forcing the decoder to rely on a single context vector, let the decoder **look back** at all encoder hidden states and **choose** which ones to attend to at each step:

$$e_{t,i} = \mathbf{v}^\top \tanh(\mathbf{W}_a\mathbf{h}_{t-1}^{\text{dec}} + \mathbf{U}_a\mathbf{h}_i^{\text{enc}}) \quad \text{(alignment score)} \tag{1.22}$$

$$\alpha_{t,i} = \frac{\exp(e_{t,i})}{\sum_{j=1}^{T}\exp(e_{t,j})} \quad \text{(attention weights)} \tag{1.23}$$

$$\mathbf{c}_t = \sum_{i=1}^{T}\alpha_{t,i}\mathbf{h}_i^{\text{enc}} \quad \text{(context vector)} \tag{1.24}$$

At each decoding step, the model computes a relevance score $e_{t,i}$ between the current decoder state and each encoder position. These scores are normalized into a probability distribution $\alpha_{t,i}$—the **attention weights**. The context $\mathbf{c}_t$ is a weighted sum of encoder states, where the weights indicate where the model is "looking."

This solved the bottleneck problem. No matter how long the input, the decoder could access any part of it at any time. The model learned to align source words with target words—a task that previously required separate, hand-crafted alignment models. The attention weights visualized what the model was "thinking about" at each step, making neural translation interpretable for the first time.

Bahdanau attention was the spark. But the fire was yet to come.

1.6.1 Luong Attention (Luong et al., 2015)

Minh-Thang Luong and colleagues at Stanford proposed a simpler, faster variant in *Effective Approaches to Attention-based Neural Machine Translation* (2015). They computed attention using only dot products, without the tanh nonlinearity:

$$e_{t,i} = (\mathbf{h}_t^{\text{dec}})^\top \mathbf{h}_i^{\text{enc}} \quad \text{(dot-product attention)} \tag{1.25}$$

They also introduced **global attention** (attend to all source positions) and **local attention** (attend only to a window around a predicted alignment position). The dot-product variant proved surprisingly effective and foreshadowed what was to come.

1.7 The Stage is Set

By 2015, all the pieces were in place. Word embeddings provided continuous representations of language. LSTMs and GRUs could process sequences while remembering long-range dependencies. The encoder-decoder architecture framed language tasks as sequence-to-sequence problems. And attention allowed models to dynamically focus on relevant parts of the input.

But these components were bolted together with LSTMs at the core. LSTMs are inherently sequential—to compute $\mathbf{h}_t$, you must first compute $\mathbf{h}_{t-1}$. This sequential bottleneck made training slow and limited how large models could grow.

The field was ready for something better. It was ready for the Transformer.

Chapter 2

The Transformer Revolution (2017–2019)

2.1 Attention Is All You Need

In the summer of 2017, eight researchers at Google published a paper with an audacious title: *Attention Is All You Need* (Vaswani et al., 2017). The paper proposed an architecture with **no recurrence whatsoever**—no LSTMs, no GRUs, no sequential processing. Just attention.

The idea was almost heretical. For decades, recurrence had been considered essential for sequence processing. The title itself was a provocation: attention, which had been treated as a mere *enhancement* to recurrent models, was now claimed to be *sufficient* on its own.

The paper was rejected from ICLR 2018. Reviewers found it "too simple" and "incrementally novel." It would go on to become one of the most cited papers in computer science history.

2.1.1 The Transformer Architecture

The Transformer consists of stacked encoder and decoder layers. But instead of recurrence, it uses a mechanism Vaswani et al. called **Scaled Dot-Product Attention**:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V} \tag{2.1}$$

Here $\mathbf{Q}$ (queries), $\mathbf{K}$ (keys), and $\mathbf{V}$ (values) are all projections of the same input sequence:

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_Q, \quad \mathbf{K} = \mathbf{X}\mathbf{W}_K, \quad \mathbf{V} = \mathbf{X}\mathbf{W}_V \tag{2.2}$$

The names are borrowed from information retrieval: you have a query, you match it against keys, and you retrieve the corresponding values. But crucially, all of this is learned—the model learns what "queries," "keys," and "values" mean for the task at hand.

The $\sqrt{d_k}$ scaling prevents the dot products from growing too large as dimensionality increases. Without it, the softmax would saturate into regions of near-zero gradients.

2.1.2 Multi-Head Attention

Rather than computing a single attention function, the Transformer uses **multi-head attention**:

$$\text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\mathbf{W}_O \tag{2.3}$$

where each head independently computes:

$$\text{head}_i = \text{Attention}(\mathbf{X}\mathbf{W}_{Q_i}, \mathbf{X}\mathbf{W}_{K_i}, \mathbf{X}\mathbf{W}_{V_i}) \tag{2.4}$$

Each head can learn to attend to different aspects of the input—one head might focus on syntactic relationships, another on semantic content, a third on positional proximity. The multi-head design gives the model multiple "perspectives" on the same sequence.

2.1.3 Positional Encoding

Without recurrence, the Transformer has no inherent notion of token order. It processes the entire sequence simultaneously—a strength for parallelism, but a problem for representing position. Vaswani et al. solved this by adding **sinusoidal positional encodings** to the input embeddings:

$$PE_{(pos,2i)} = \sin(pos/10000^{2i/d_{\text{model}}}) \tag{2.5}$$

$$PE_{(pos,2i+1)} = \cos(pos/10000^{2i/d_{\text{model}}}) \tag{2.6}$$

These encodings create a unique "position signature" for each token, using sinusoids at different frequencies. The choice of sinusoids was deliberate: for any fixed offset k, PE_{pos+k} can be expressed as a linear function of PE_{pos}, which the authors hypothesized would help the model learn relative positions.

2.1.4 Why the Transformer Destroyed the LSTM

The Transformer had three killer advantages:

1. Parallelism. An LSTM processes tokens sequentially: $\mathbf{h}_1 \to \mathbf{h}_2 \to \ldots \to \mathbf{h}_T$. A Transformer processes all tokens simultaneously, since attention between any pair (i, j) can be computed independently. Training time dropped from days to hours.

2. Long-range dependencies. In an LSTM, information must travel through $O(T)$ sequential steps to connect distant tokens. In a Transformer, the path length between any two tokens is $O(1)$—they interact directly through attention. This made Transformers vastly better at modeling long-range dependencies.

3. Interpretability. Attention weights α_{ij} show exactly which tokens each position is "looking at." This transparency made Transformers easier to analyze and debug than the opaque hidden states of LSTMs.

The Transformer achieved state-of-the-art results on WMT 2014 English-to-German (28.4 BLEU, beating the previous best ensemble by 2.0 BLEU) and English-to-French (41.8 BLEU, best single model). But these numbers undersold the revolution. The Transformer was not just a better translation model—it was a *universal sequence processor* that could be applied to any domain where attention made sense.

2.2 GPT-1: Generative Pre-Training

In June 2018, Alec Radford and his colleagues at OpenAI published a technical report that would define the next era: *Improving Language Understanding by Generative Pre-Training* (Radford et al., 2018). They called their model **GPT**.

The key insight was architectural. The original Transformer used both an encoder and a decoder. For translation, this made sense: you read the source with the encoder, then generate the target with the decoder, attending to the encoder outputs.

But Radford et al. asked: what if we just want to generate text? They stripped away the encoder entirely, leaving only the **decoder**. The result was a 12-layer, 117-million-parameter model that did one thing: predict the next token.

$$\mathcal{L}_{\text{pre-train}} = \sum_i \log P(u_i \mid u_{i-k}, \ldots, u_{i-1}; \Theta) \tag{2.7}$$

This was language modeling in its purest form. No task-specific architectures. No hand-crafted features. Just: given a sequence of text, what comes next?

But here was the twist. After pre-training on 7,000 books (the BooksCorpus), Radford et al. showed that this simple model could be **fine-tuned** for almost any NLP task. Sentiment analysis? Feed in "The movie was [MASK]," fine-tune, and classify the output. Question answering? Feed in "Context: X. Question: Y. Answer: [MASK]." Natural language inference? Same pattern.

A single architecture, a single pre-training objective, and a simple fine-tuning recipe produced state-of-the-art results on 9 out of 12 tasks (including RACE reading comprehension: 72.8 vs. previous 59.0).

The **decoder-only pre-training paradigm** was born. And it would dominate everything that followed.

2.3 BERT: The Bidirectional Counterpart

In October 2018, Jacob Devlin and colleagues at Google published *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding* (Devlin et al., 2019).

Where GPT was a decoder-only model, BERT was **encoder-only**. This meant BERT could see the entire input simultaneously—both the words before AND after each token. For understanding tasks (classification, QA, NER), this bidirectional context was crucial.

BERT's training was brilliantly simple. Take a sentence. Randomly mask 15% of the words. Ask the model to predict the masked words based on the surrounding context. This was the **Masked Language Modeling (MLM)** objective:

$$\mathcal{L}_{\text{MLM}} = -\sum_{i \in \text{masked}} \log P(w_i \mid w_{\text{unmasked}}) \tag{2.8}$$

BERT also included a **Next Sentence Prediction (NSP)** objective: given two sentences, predict whether the second follows the first. This was designed to help with tasks requiring sentence-level reasoning.

The results were staggering. BERT-Large (340M parameters) achieved 7.7% absolute improvement on GLUE over the previous state of the art. It scored 93.2 F1 on SQuAD v1.1, pushing QA performance to levels previously thought unattainable.

BERT established the **pre-train then fine-tune** paradigm as the standard for NLP. But in an ironic twist, the BERT family would ultimately be eclipsed by the GPT approach. Why? Because BERT cannot generate text. It can understand language, but it cannot produce it. GPT can do both. And as models grew larger, the decoder-only approach proved more scalable and more general.

2.4 GPT-2: The Power of Scale

On February 14, 2019, OpenAI published *Language Models are Unsupervised Multitask Learners* (Radford et al., 2019), introducing GPT-2. At 1.5 billion parameters—a 12× scale-up from GPT-1—it was the largest language model yet built.

But the paper's key claim was more audacious than the model's size. Radford et al. argued that a sufficiently large language model could perform tasks **without any fine-tuning at all**. They called this **zero-shot transfer**.

The idea was simple: instead of fine-tuning on task-specific data, you simply tell the model what you want in natural language. "Translate to French: Hello world →" and the model completes the translation. "Summary: [long text] TL;DR:" and the model summarizes.

GPT-2 achieved 63.1 F1 on CoQA (reading comprehension) without ever being trained on QA data. It scored 55.5 BLEU on WMT-14 French-to-English translation without seeing parallel text. These weren't state-of-the-art numbers, but they were *possible at all*—and they suggested that scale alone might unlock capabilities no one had designed for.

OpenAI was sufficiently concerned about misuse that they initially declined to release the full 1.5B model, citing risks of AI-generated disinformation. This sparked a heated debate about openness, safety, and the responsibilities of AI researchers. The model was eventually released in stages over eight months.

GPT-2 also made an architectural contribution that would become standard: it moved layer normalization to the **input** of each sub-block (pre-normalization), added a final layer norm, and modified weight initialization to scale residuals by $1/\sqrt{N}$. These engineering improvements enabled training deeper models reliably.

2.4.1 RoBERTa: BERT, But Better

Also in 2019, Yinhan Liu and colleagues at Facebook AI published *RoBERTa: A Robustly Optimized BERT Pretraining Approach* (Liu et al., 2019). Their contribution was not architectural but methodological. They showed that BERT had been **significantly undertrained**:

- Trained on 10× more data (160 GB vs. 16 GB).
- Used **dynamic masking** (different masks each epoch) instead of static masking.
- Removed the NSP objective entirely—it wasn't helping.
- Larger batches (8K sequences) and longer training.

With these changes, the same BERT architecture achieved 88.5 on GLUE (vs. 83.2 for BERT-Large). The lesson was profound: **training methodology matters as much as architecture**. A well-trained small model often outperforms a poorly-trained large one—a lesson that would be rediscovered many times.

2.5 The GPT vs. BERT Divergence

By the end of 2019, two distinct paradigms had emerged:

The BERT path (encoder-only): Pre-train on MLM. Fine-tune for each downstream task. Excellent for NLU. Cannot generate text.

The GPT path (decoder-only): Pre-train on next-token prediction. Use zero-shot or few-shot prompting. Can both understand AND generate.

The GPT path would win. Not because BERT was worse at understanding—it wasn't. But because the future of AI was generation. A model that can generate text can also, via clever prompting, understand it. The reverse is not true. When ChatGPT arrived in 2022, it was built on the GPT path, not the BERT path.

The stage was set for an explosion of scale. And for a question that would define the next chapter: how big can we go, and what happens when we get there?

Chapter 3

The Age of Scale (2020–2022)

3.1 GPT-3: In-Context Learning Emerges

In May 2020, OpenAI published a paper that dwarfed everything before it. *Language Models are Few-Shot Learners* (Brown et al., 2020) introduced GPT-3, a 175-billion-parameter model trained on 570 GB of text. It was 116× larger than GPT-2.

The architectural changes from GPT-2 were minimal. GPT-3 used the same decoder-only transformer, with one twist: it alternated between dense attention and locally banded sparse attention in different layers (inspired by the Sparse Transformer, Child et al., 2019). But the real innovation was not architectural. It was behavioral.

GPT-3 demonstrated something no one had predicted: **in-context learning**. Given a few examples in the prompt—no gradient updates, no fine-tuning—the model could learn to perform entirely new tasks:

- With 64 examples of English-to-French translation, GPT-3 achieved 35.8 BLEU.
- With 50 examples of arithmetic word problems, it solved 65% correctly.
- Given a description of a made-up word, it could use it correctly in a sentence.

The performance improved smoothly with model size, following a power law. There was no sudden jump, no "emergence" at a critical threshold—just steady, predictable improvement as parameters increased.

But in that steady improvement lay a profound implication: **the model was learning to learn**. Each example in the prompt was teaching it a pattern, and it was applying that pattern to new instances. This was **meta-learning**—the ability to acquire new skills from demonstrations alone.

GPT-3 also demonstrated alarming capabilities. It could generate coherent news articles that human evaluators found difficult to distinguish from human-written ones (52% detection rate vs. 50% random). It could write functional code from natural language descriptions. It could compose poetry, draft legal documents, and role-play characters.

The cost of this capability was staggering. Training GPT-3 cost an estimated $4.6 million in compute, using thousands of GPUs over months. Inference was also expensive: a single query to the 175B model required significant compute. The economics of scale were becoming a barrier to entry.

3.2 Scaling Laws: The Science of Bigger

As models grew, a critical question emerged: how should we allocate our compute budget? Should we build bigger models or train on more data?

3.2.1 Kaplan et al. (2020): The First Scaling Laws

Jared Kaplan and his collaborators at OpenAI published *Scaling Laws for Neural Language Models* (Kaplan et al., 2020), providing the first quantitative answer. They trained models across a wide range of sizes and data budgets, and fitted power laws:

$$L(N) = \left(\frac{N_c}{N}\right)^{\alpha_N}, \quad L(D) = \left(\frac{D_c}{D}\right)^{\alpha_D} \tag{3.1}$$

Their key finding: **model size should grow faster than data size**. For a given increase in compute, you should allocate most of the budget to bigger models, not more training data:

$$N_{\text{opt}}(C) \propto C^{0.73}, \quad D_{\text{opt}}(C) \propto C^{0.27} \tag{3.2}$$

This finding shaped the industry for two years. It justified the "big models" approach: if you have more compute, build a bigger model. GPT-3's 175B parameters with "only" 300B tokens was consistent with these laws.

3.2.2 Chinchilla (Hoffmann et al., 2022): The Correction

In March 2022, Jordan Hoffmann and colleagues at DeepMind published a paper that turned the scaling laws upside down. *Training Compute-Optimal Large Language Models* (Hoffmann et al., 2022) found that Kaplan et al. had **systematically undertrained their large models**.

The Chinchilla team trained over 400 models at different sizes and data budgets, with careful controls that Kaplan et al. had not applied (cosine learning rate schedules, consistent hyperparameter methodology). Their revised scaling law:

$$N_{\text{opt}}(C) \propto C^{0.49}, \quad D_{\text{opt}}(C) \propto C^{0.51} \tag{3.3}$$

The shocking implication: **parameters and tokens should grow at roughly equal rates**. For a compute budget of 6×10^{23} FLOPs, the optimal allocation was 70B parameters trained on 1.4T tokens—not 280B parameters on 300B tokens.

To prove their point, they trained **Chinchilla**, a 70B model on 1.4T tokens. Despite being 4× smaller than DeepMind's own Gopher (280B parameters), Chinchilla outperformed it across the board. It even beat GPT-3 (175B) while being less than half the size.

The lesson was clear and painful: **the field had been massively undertraining its models**. GPT-3, with only ∼1.7 tokens per parameter, was nowhere near compute-optimal. The Chinchilla ratio of ∼20 tokens per parameter became the new standard, and every major lab retooled their training pipelines.

3.3 InstructGPT: Teaching Models to Be Helpful

GPT-3 was brilliant. It was also, frequently, terrible. It would fabricate facts with startling confidence, generate toxic or biased content, and stubbornly refuse to follow simple instructions. The problem was not capability—it was **alignment**. The model had been trained to predict internet text, not to be helpful, honest, and harmless.

In January 2022, Long Ouyang and colleagues at OpenAI published *Training Language Models to Follow Instructions with Human Feedback* (Ouyang et al., 2022), introducing **InstructGPT** and the **RLHF** (Reinforcement Learning from Human Feedback) pipeline.

The pipeline had three stages:

Stage 1—Supervised Fine-Tuning (SFT): Human annotators wrote ideal responses to a diverse set of prompts (∼13K). The model was fine-tuned on these demonstrations.

Stage 2—Reward Model (RM) Training: For each prompt, the model generated multiple responses. Human annotators ranked them from best to worst. A reward model $r_\phi(x, y)$ was trained to predict these preferences:

$$\mathcal{L}_{\text{RM}} = -\mathbb{E}_{(x,y_w,y_l)} \left[\log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l))\right] \tag{3.4}$$

This is the Bradley-Terry model of pairwise preferences, expressed as a binary cross-entropy loss.

Stage 3—PPO Optimization: The SFT model was further optimized using Proximal Policy Optimization (Schulman et al., 2017), with the reward model providing the signal:

$$\max_\theta \mathbb{E}_{x,y\sim\pi_\theta} \left[r_\phi(x, y) - \beta \cdot \text{KL}(\pi_\theta(\cdot|x) \| \pi_{\text{ref}}(\cdot|x))\right] \tag{3.5}$$

The KL penalty prevented the model from diverging too far from its SFT behavior, preserving its general capabilities while improving alignment.

The results were striking: **human evaluators preferred InstructGPT-1.3B over GPT-3 175B**. A model 100× smaller, but aligned with human preferences, was judged more useful than its massive unaligned cousin. Alignment mattered more than scale.

3.4 The Emergence Phenomenon

As models grew past certain thresholds, researchers observed something unsettling: capabilities appeared suddenly, without warning. A model would score near-random on a task at 10B parameters, then jump to near-perfect at 100B. This was **emergence**—qualitative changes in behavior arising from quantitative increases in scale.

The paper that crystallized this observation was *Emergent Abilities of Large Language Models* (Wei et al., 2022), which surveyed the landscape of emergent behaviors. At small scales, models couldn't do multi-step arithmetic, couldn't follow chain-of-thought reasoning, couldn't use tools, couldn't self-correct. Then, somewhere between 10B and 100B parameters, these abilities appeared—not gradually, but discontinuously.

This had profound implications. It meant that you couldn't predict what a model would be capable of by studying smaller versions. It meant that safety testing at sub-scale might miss critical risks. And it meant that the path to AGI might involve crossing thresholds we cannot anticipate.

3.5 ChatGPT: The World Takes Notice

On November 30, 2022, OpenAI released a chatbot built on GPT-3.5 and fine-tuned with RLHF. They called it **ChatGPT**. Within five days, it had one million users. Within two months, it had 100 million users—the fastest-growing consumer application in history.

ChatGPT was not technically novel. It combined GPT-3.5's architecture with InstructGPT's alignment pipeline. The conversational interface, the refusal of harmful requests, and the multi-turn memory were all refinements of existing techniques.

But ChatGPT did something no previous AI system had done: it **felt** intelligent to ordinary people. It could explain quantum physics at a fifth-grade level. It could debug code and explain the fix. It could write a sonnet about a programmer's love for coffee. It could role-play as a pirate, a therapist, or a Linux terminal.

The world reacted with a mixture of wonder and panic. Schools banned it. Companies raced to integrate it. Pundits declared the end of homework, the death of the essay, the obsolescence of search engines. OpenAI's valuation soared toward $30 billion. Microsoft invested $10 billion and began integrating GPT-4 into every product.

The AI summer had arrived. But the ecosystem was about to fracture in ways no one expected.

Chapter 4

The Open Revolution (2023)

4.1 The Closed Frontier

At the start of 2023, the LLM landscape was a monopoly. OpenAI's GPT-3.5 and GPT-4 dominated through their API, with Google's PaLM and DeepMind's Chinchilla available only to select partners. Anthropic's Claude was invite-only. The frontier was closed.

Researchers could query these models, but they could not see inside them. They could not study their weights, their training data, or their failure modes. This was a crisis for the scientific community. How do you study a black box? How do you build on technology you cannot access?

The answer came from an unexpected source. And it would change everything.

4.2 LLaMA: The Leak That Democratized AI

On February 24, 2023, Meta released **LLaMA** (*Large Language Model Meta AI*) in a paper by Hugo Touvron and colleagues (Touvron et al., 2023). Unlike OpenAI's models, LLaMA's weights were made available to researchers upon request.

The architecture was built on lessons learned from years of community research. Every component was chosen for efficiency:

- **RMSNorm** instead of LayerNorm—30% faster normalization.
- **SwiGLU activation** (Shazeer, 2020) instead of ReLU—better gradient flow.
- **RoPE positional embeddings** (Su et al., 2021) instead of learned positions—better length generalization.

But LLaMA's most profound choice was its training data. Rather than training on a massive but compute-optimal mix, Meta trained on 1.4T tokens of heavily filtered web text, books, academic papers, and code. This was **quality over quantity**—and it worked.

LLaMA-13B outperformed GPT-3 (175B) on most benchmarks. LLaMA-33B and 65B were competitive with PaLM (540B). A research team at Meta, with a relatively modest compute budget, had built open models that rivaled the closed frontier.

Then, on March 2, 2023, LLaMA's weights leaked online. Anyone with a torrent client could download them. The floodgates opened.

4.3 The Fine-Tuning Explosion

Within weeks of the LLaMA leak, a Cambrian explosion of fine-tuned models erupted from the open-source community:

Vicuna (Chiang et al., March 2023): Researchers at LMSYS fine-tuned LLaMA on 70,000 conversations from ShareGPT. Training cost: $300 on 8× A100 GPUs. The result? A chatbot that GPT-4 rated as 90% of ChatGPT quality.

WizardLM (Xu et al., April 2023): Microsoft researchers introduced **Evol-Instruct**, a method that iteratively increases instruction complexity. Starting from simple prompts, a strong LLM generates progressively more complex instructions and their responses. Fine-tuning on this evolved dataset produced WizardLM, which matched ChatGPT on some tasks.

Alpaca (Taori et al., March 2023): Stanford researchers fine-tuned LLaMA-7B on 52,000 instruction-following demonstrations generated by GPT-3.5. Training cost: less than $600. The model demonstrated that instruction-following can be "distilled" from a strong teacher to a weak student.

Orca (Mukherjee et al., June 2023): Microsoft researchers went further, distilling not just answers but **step-by-step reasoning traces** from GPT-4. They showed that training on explanations, not just outputs, produces dramatically better students.

The lesson was unmistakable: **post-training matters more than pre-training for user-facing quality**. A well-fine-tuned 7B model could outperform a poorly-tuned 70B model. The community had discovered that alignment, instruction-following, and conversation skills could be achieved with modest compute—if you had good data and clever methods.

4.4 LLaMA 2: Open and Aligned

In July 2023, Meta responded to the community's efforts with **LLaMA 2** (Touvron et al., 2023). This time, the weights were openly released, not leaked. The paper ran to 77 pages and detailed every aspect of training and alignment.

LLaMA 2's 70B model introduced **Grouped Query Attention (GQA)**, where 8 KV heads serve 64 query heads. This reduced KV-cache memory by 8× while retaining 97%+ of MHA quality. The 7B and 13B models retained standard MHA.

More importantly, LLaMA 2 came with **LLaMA 2-Chat**, a fully aligned model trained through an elaborate multi-stage pipeline. The alignment involved:

1. **SFT** on publicly available instruction data.
2. **RLHF** with **two reward models**—one for helpfulness, one for safety. The PPO objective used whichever score was lower, implementing a pessimistic safety-first approach.
3. **Ghost Attention (GAtt)**—a technique that reinforces system prompts by prepending them to every user turn during training, preventing the model from forgetting its instructions across multi-turn conversations.
4. **Five rounds of iterative refinement** with weekly human feedback collection.

The result was the first open model that was both highly capable and reasonably aligned. LLaMA 2-Chat 70B was comparable to ChatGPT in human evaluations. And it was free to download, modify, and deploy.

4.5 Mistral: European Efficiency

In September 2023, a French startup called Mistral AI released **Mistral 7B** (Jiang et al., 2023), a model that matched LLaMA-2 13B despite being nearly half the size.

Mistral's architecture was a masterclass in efficiency:

- **Sliding Window Attention**: Each token attends only to the 4,096 most recent tokens. This gives constant KV-cache memory regardless of sequence length. The effective context grows with depth—layer ℓ can reach $\sim \ell \times 4096$ tokens back through the layer stack.
- **Grouped Query Attention**: 32 query heads, 8 KV heads.
- **Rolling Buffer KV-Cache**: A circular buffer stores only the last window of key-value pairs. No memory growth with longer sequences.

Mistral 7B achieved what many thought impossible: a 7B model that could compete with 13B models across the board. Its Instruct variant, trained with simple SFT (no RLHF), demonstrated that even basic fine-tuning could produce strong instruction-following.

Then, in December, Mistral dropped a bombshell: **Mixtral 8×7B** (Jiang et al., 2024), a Mixture of Experts model. It had 46.7B total parameters but only 12.9B active per token, because each token activated only 2 of 8 experts. The model matched or exceeded LLaMA-2 70B on most benchmarks, with 5× fewer active parameters.

Mixtral was the first open MoE model to achieve frontier performance. It introduced a key design choice: MoE layers replaced the FFN only in **alternating** transformer layers, with dense attention layers between them. This preserved representational continuity while enabling expert specialization. Remarkably, Mixtral used **no auxiliary load-balancing loss**—the router naturally distributed tokens evenly without explicit encouragement.

4.6 DPO: The End of RLHF

In May 2023, Rafael Rafailov and colleagues at Stanford published a paper that simplified alignment dramatically. *Direct Preference Optimization* (DPO; Rafailov et al., 2023) eliminated the need for reward models and PPO entirely.

The key insight was mathematical. Under the KL-constrained RLHF objective, the optimal policy π^* has a closed-form relationship with the reward function:

$$r(x, y) = \beta \log \frac{\pi^*(y|x)}{\pi_{\text{ref}}(y|x)} + \beta \log Z(x) \tag{4.1}$$

By substituting this into the Bradley-Terry preference model, the partition function $Z(x)$ cancels out, yielding a direct loss on preference pairs:

$$\mathcal{L}_{\text{DPO}} = -\mathbb{E}_{(x,y_w,y_l)} \left[\log \sigma \left(\beta \log \frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \right) \right] \tag{4.2}$$

This is stunningly simple. No reward model. No PPO. No on-policy sampling. No KL penalty engineering. Just a supervised-style loss that increases the probability of preferred responses and decreases the probability of dispreferred ones.

Zephyr (Tunstall et al., October 2023), built by HuggingFace, demonstrated DPO's power. Starting from Mistral 7B, they applied SFT on synthetic dialogues, then DPO

using GPT-4 as judge. The result was Zephyr-7B-β, which matched LLaMA-2-Chat 70B on MT-Bench. A 7B model, through careful alignment, matching a 70B model.

DPO democratized alignment. You no longer needed a GPU farm to run PPO. You needed preference data and a few hours of fine-tuning.

4.7 The State of Play: December 2023

By the end of 2023, the landscape had transformed completely. A year earlier, the frontier was closed. Now:

- LLaMA 2 70B provided GPT-3.5-class open weights with alignment.
- Mistral 7B and Mixtral 8×7B offered efficiency-first alternatives.
- DPO had simplified alignment from a three-stage RL pipeline to a single loss function.
- The fine-tuning ecosystem (Vicuna, WizardLM, Orca, Zephyr) had shown that post-training quality matters as much as pre-training scale.
- A global community of researchers, hobbyists, and startups was building on open models faster than any single company could have managed alone.

The open revolution had succeeded. The frontier was no longer a walled garden. But the next year would bring even more dramatic changes—in architecture, in efficiency, and in what models could do.

Chapter 5

Efficiency and Specialization (2024)

5.1 DeepSeek: The Efficiency Manifesto

If 2023 was the year of open weights, 2024 was the year of DeepSeek. The Chinese AI lab published a series of papers that systematically reimagined every aspect of LLM architecture for efficiency.

5.1.1 DeepSeek LLM (January 2024): The Longtermist Philosophy

DeepSeek's first major paper, *DeepSeek LLM: Scaling Open-Source Language Models with Longtermism* (DeepSeek-AI, 2024), introduced a philosophy that would define their approach: train on as much data as possible, even beyond Chinchilla-optimal ratios. Their 67B model, trained on 2T tokens, matched LLaMA-2 70B. But the paper's real contribution was demonstrating that **data scaling continues to pay off** far beyond what Chinchilla predicted.

5.1.2 DeepSeekMoE (January 2024): Fine-Grained Experts

The same month, DeepSeek published *DeepSeekMoE: Towards Ultimate Expert Specialization* (Dai et al., 2024), which rethought the Mixture of Experts architecture:

- **Shared experts** ($K_s = 2$) process every token, capturing common knowledge.
- **Routed experts** ($K_r = 64$ to 160) are selectively activated.
- **Fine-grained segmentation**: instead of 8 large experts, use 64+ smaller experts. Each expert becomes a narrower specialist.

The shared+routed decomposition prevents knowledge redundancy (where multiple experts learn the same thing) and knowledge hybridity (where a single expert handles too many unrelated patterns). It would become the template for efficient MoE.

5.1.3 DeepSeek-V2 (May 2024): Multi-Head Latent Attention

In May, DeepSeek unveiled their masterpiece: *DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model*. The paper introduced **Multi-Head Latent Attention (MLA)**, which solved the KV-cache memory problem:

$$\mathbf{c}_t^{KV} = \mathbf{x}_t \mathbf{W}^{DKV} \in \mathbb{R}^{d_c} \quad \text{(compressed latent, } d_c \ll d\text{)} \tag{5.1}$$

$$\mathbf{k}_t^{(i)} = \mathbf{c}_t^{KV}\mathbf{W}_i^{UK}, \quad \mathbf{v}_t^{(i)} = \mathbf{c}_t^{KV}\mathbf{W}_i^{UV} \tag{5.2}$$

Instead of caching the full-dimensional keys and values for every head at every layer, MLA compresses them into a low-rank latent vector—and caches only that. Per-head K and V are reconstructed on-the-fly during attention. For a model with $h = 128$ heads and $d = 8192$, this reduces KV-cache memory by an order of magnitude.

Combined with DeepSeekMoE, the result was a 236B model where only 21B parameters activate per token—and the KV-cache was compressed by 90%+. Training cost: $5 million. API pricing: 1/10th of GPT-4. The economics of frontier AI had shifted.

5.1.4 DeepSeek-V3 (December 2024): Multi-Token Prediction

The final paper in the sequence, released on December 26, 2024, scaled everything to the extreme. *DeepSeek-V3* (DeepSeek-AI, 2024) was a 671B MoE model with 37B active parameters. It introduced three major innovations:

1. Multi-Token Prediction (MTP): Instead of predicting only the next token, the model predicts the next n tokens (with $n = 2$). Additional prediction heads at the final layer generate subsequent tokens. The training loss combines all predictions:

$$\mathcal{L} = \sum_t \sum_{i=0}^{n-1} \lambda_i \cdot \log p(y_{t+i} \mid y_{<t}) \tag{5.3}$$

This provides a richer training signal and enables **speculative decoding** at inference—the multi-token predictions serve as draft tokens, verified by the main model.

2. Auxiliary-Loss-Free Load Balancing: Instead of using auxiliary losses to balance expert usage (which compete with the primary objective), DeepSeek-V3 dynamically adjusts per-expert bias terms. After each training step, overloaded experts have their bias reduced; underloaded experts have theirs increased. This achieves load balancing without gradient interference.

3. FP8 Mixed Precision Training: GEMM operations run in FP8 (E4M3 format) with block-wise quantization. This doubles training throughput and reduces memory by 40%. The model was trained on 2048 H800 GPUs over two months at a total cost of $5.576M.

The result was the most capable fully open model to date. DeepSeek-V3 matched GPT-4o and Claude 3.5 Sonnet on most benchmarks, at 1/20th to 1/50th the inference cost. The economics of AI had been rewritten.

5.2 LLaMA 3: Meta's Counterpunch

In April 2024, Meta released **LLaMA 3**, their answer to the rapidly evolving open-source landscape. The 8B and 70B models were trained on 15T tokens—4× more than LLaMA 2. The tokenizer expanded from 32K to 128K tokens. RoPE's base frequency jumped from 10,000 to 500,000. And GQA was now used at all sizes.

Then, in July 2024, Meta dropped **LLaMA 3.1 405B**—the first truly frontier-class open model. At 405 billion dense parameters, trained on 15T tokens with a 128K context window, it matched GPT-4 across the board. It was the largest openly available model, and it was competitive with the best closed models in the world.

The alignment strategy had also evolved. LLaMA 3 used **DPO** instead of RLHF PPO, following the community's lead. The alignment proceeded through four rounds:

SFT on human annotations, SFT on synthetic data from rejection sampling, DPO on preference data, and a final best-of-N rejection sampling stage. The result was LLaMA 3.1 Instruct, which scored among the top models on LMSys Chatbot Arena.

LLaMA 4, announced in April 2025, would take the MoE route. The Scout variant (17B active, 109B total) and Maverick variant (17B active, 400B total) demonstrated Meta's commitment to efficient architectures.

5.3 The Small Model Renaissance

While frontier models pushed toward 400B+ parameters, a parallel movement argued that small could be mighty.

5.3.1 Phi: The Data Quality Hypothesis

Microsoft's Phi series (Gunasekar et al., 2023; Abdin et al., 2024) demonstrated that **training data quality** is a substitute for scale. Phi-1 (1.3B parameters) was trained on 1B tokens of **synthetic textbook-quality data**—code exercises and explanations generated by GPT-3.5. It scored 50.6% on HumanEval, beating models 10× its size.

Phi-2 (2.7B) extended this to natural language reasoning, outperforming 7B models. Phi-3 (3.8B) matched LLaMA-3 8B on many benchmarks. The secret was not architecture but **data curriculum**: generate high-quality synthetic data from large models, filter ruthlessly for quality, and train small models on the best examples.

5.3.2 Gemma 2: Architectural Innovation at Small Scale

Google's Gemma 2 (June 2024) took a different approach: **architectural innovation** to compensate for small scale. The 27B model used:

- **Interleaving local-global attention**: even layers use sliding window (local), odd layers use full attention (global). This reduces compute while maintaining quality.
- **Logit soft-capping**: $\text{logit}_{\text{capped}} = c \cdot \tanh(\text{logit}/c)$ with $c = 30$. Prevents the model from becoming overconfident, improving calibration and training stability.
- **Pre-norm + post-norm sandwich**: Both pre- and post-attention normalization.

Gemma 2 27B outperformed LLaMA-3 70B on LMSys Chatbot Arena while being 2.6× smaller.

5.4 Mamba: The Return of Recurrence

In December 2023, Albert Gu and Tri Dao published *Mamba: Linear-Time Sequence Modeling with Selective State Spaces* (Gu & Dao, 2023), proposing an architecture that challenged attention itself.

Mamba is a **State Space Model (SSM)**—an architecture with roots in control theory. Unlike attention (which costs $O(L^2)$ in sequence length), SSMs cost $O(L)$. But previous SSMs were **content-agnostic**: they processed every token identically, unable to decide what to remember.

Mamba's key innovation was **selectivity**: the SSM parameters become functions of the input:

$$\mathbf{B}_t = \text{Linear}_B(\mathbf{x}_t), \quad \mathbf{C}_t = \text{Linear}_C(\mathbf{x}_t), \quad \Delta_t = \text{softplus}(\text{Linear}_\Delta(\mathbf{x}_t)) \tag{5.4}$$

This allows the model to **selectively remember** important tokens and forget irrelevant ones—like an LSTM, but trained in parallel. A Mamba model with 2.8B parameters matched transformer models at twice the size.

Mamba-2 (Dao & Gu, May 2024) proved a deeper connection: SSMs and linear attention are two views of the same computation, connected through structured (semi-separable) matrices. By reformulating SSMs as block-decomposed matrix multiplication, Mamba-2 achieved 2–8× speedup and could freely interleave with attention layers.

Jamba (AI21 Labs, March 2024) combined Mamba, attention, and MoE in a single model. **Falcon-Mamba 7B** (TII, 2024) proved that a pure SSM model could compete at 7B scale. The **RWKV** series (Peng et al., 2023–2024) pursued a parallel path, blending RNN-like recurrence with transformer-like parallel training.

The architectural monoculture was breaking. For the first time since 2017, attention had serious competition.

5.5 GPT-4o and Claude 3.5: The Multimodal Frontier

While the open ecosystem flourished, the closed frontier advanced on a different axis: **multimodality**.

In May 2024, OpenAI released **GPT-4o** ("o" for omni), a model that natively processed text, images, and audio in a single unified architecture. It could respond to voice input in 232 milliseconds on average, matching human conversational latency. The model could perceive speaker emotion, generate expressive speech, and reason across modalities—not through separate pipelines, but through a single neural network.

In June 2024, Anthropic released **Claude 3.5 Sonnet**, which scored 92.0% on HumanEval (coding) and 59.4% on GPQA (graduate-level reasoning). Claude 3.5 was not a single model but a family—Haiku (fast), Sonnet (balanced), and the forthcoming Opus (most capable). In October 2024, Anthropic demonstrated **Computer Use**: a Claude variant that could view screenshots, move a mouse, click buttons, and type text—controlling a computer like a human would.

The frontier was now multimodal, conversational, and agentic. And the next frontier—reasoning—was about to be breached.

Chapter 6

Reasoning and Agency (2024–2025)

6.1 The Reasoning Gap

By mid-2024, LLMs had mastered language. They could summarize, translate, converse, and code. But they struggled with **reasoning**—the kind of step-by-step logical thinking that humans use to solve math problems, prove theorems, and debug complex systems.

The problem was not capability. GPT-4 and Claude 3.5 could reason when prompted to think step-by-step. The problem was reliability: they would often skip steps, make arithmetic errors, or lose the thread of a long chain of thought. They needed to learn **how** to reason, not just **that** reasoning sometimes helps.

6.2 OpenAI o1: Reasoning at Inference Time

In September 2024, OpenAI released **o1** (previously codenamed "Strawberry"), a model designed specifically for reasoning. The technical details were sparse—OpenAI's paper was characteristically opaque—but the core insight was clear: **reasoning can be improved by spending more compute at inference time.**

Previous models generated text in a single forward pass: prompt in, response out. o1 introduced a new paradigm: the model generates a **long internal chain of thought** before producing its final answer. This chain of thought is not shown to the user but is used by the model to refine its answer through self-verification and backtracking.

The results were dramatic. On the International Mathematical Olympiad qualifying exam (AIME), o1 scored in the top 500 students nationally. On GPQA (graduate-level physics, biology, chemistry), it exceeded PhD-level human performance. On Codeforces programming competitions, it reached the 89th percentile.

The key innovation was **test-time compute scaling**: rather than spending more compute on training a bigger model, spend more compute at inference time generating and evaluating multiple reasoning paths. This introduced a new axis of scaling—one that was independent of model size.

o3 (December 2024) pushed this further, scoring 25.2% on FrontierMath (a benchmark of extremely hard math problems) and 87.5% on ARC-AGI (a benchmark designed to resist memorization). These were leaps that no one had predicted.

6.3 DeepSeek-R1: Pure RL Thought

In January 2025, DeepSeek published a paper that astonished the research community. *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning* (DeepSeek-AI, 2025) demonstrated that **reasoning behavior can emerge purely from reinforcement learning, without any supervised data**.

6.3.1 R1-Zero: The Autonomous Reasoner

The experiment was audacious. DeepSeek took their base model (DeepSeek-V3-Base), with **no instruction tuning, no SFT**, and applied GRPO (Group Relative Policy Optimization) with simple rule-based rewards:

- For math problems: binary reward for correct final answer.
- For code: compiler-based reward (code compiles and passes tests).
- For format: reward for placing reasoning in `<think></think>` tags and answers in `<answer></answer>` tags.

That was it. No human demonstrations. No teacher model. No chain-of-thought examples. Just the model, the problems, and binary rewards.

What happened next was remarkable. As training progressed, the model **spontaneously developed sophisticated reasoning behaviors**:

- **Self-verification**: "Wait, is this correct? Let me check..."
- **Reflection**: Re-evaluating earlier steps and correcting errors mid-generation.
- **Alternative approaches**: "Let me try a different method..."
- Growing reasoning chains from a few lines to thousands of tokens.

At a specific point in training, the DeepSeek team observed what they called an **"aha moment"**—the model learned to backtrack and correct itself during generation. This was not programmed. It was not taught through examples. It emerged from the optimization process.

R1-Zero scored 71.0% on AIME 2024 with no SFT whatsoever. Pure RL had induced chain-of-thought reasoning.

6.3.2 R1: Polishing the Diamond

R1-Zero was messy. Its reasoning was often incomprehensible, mixing languages and producing garbled chains of thought. DeepSeek addressed this with a **cold-start SFT** phase:

1. Collect a few thousand high-quality chain-of-thought examples from R1-Zero (filtering for readability and correctness). Human annotators refine these.
2. Fine-tune V3-Base on this cold-start data. This teaches proper format and readability, without teaching *how* to reason (the model already learned that from RL).
3. Apply GRPO again on the cold-started model, now with an additional **language consistency reward** (penalize mixing languages).
4. Generate 600K reasoning traces via rejection sampling from the RL-trained model. Add 200K general traces. Fine-tune V3-Base on the combined 800K dataset.
5. Final RL stage with diverse rewards: helpfulness RM + harmlessness RM + rule-based reasoning RM.

The result was DeepSeek-R1, which scored 79.8% on AIME 2024. It matched OpenAI's o1 on reasoning benchmarks while being fully open-weight.

DeepSeek also **distilled** R1's reasoning into smaller models (Qwen-1.5B through LLaMA-70B) using the 800K reasoning traces as training data. The distilled 7B model outperformed most 70B+ models on reasoning tasks. The distilled 32B model was competitive with the frontier. **Reasoning capability had been shown to transfer from large models to small ones.**

6.4 The New Architecture of Thought

The o1 and R1 papers, taken together, revealed a new paradigm for AI capability:

1. **Pre-training** provides a base model with broad knowledge and language skills.
2. **Reasoning training** (RL or supervised) teaches structured chain-of-thought.
3. **At inference time,** the model spends additional compute on internal reasoning before answering.
4. **The amount of inference compute** can be scaled to improve answer quality—independent of model size.

This was a fundamental shift. Previous scaling had focused on training time: bigger models, more data, more GPUs. The new paradigm added a second axis: **inference-time compute**. You could take the same model and, by allowing it to "think longer," get better answers.

This had profound implications. It meant that deploying a capable AI system was no longer just about having the most GPUs for training. It was also about designing systems that could allocate inference compute intelligently—spending more on hard problems, less on easy ones. The compute budget became a **stratified resource**, allocated across problems based on difficulty.

6.5 Toward Agency

In 2024 and 2025, the frontier expanded beyond reasoning into the domain of **agency**—the ability to take actions in the world, not just produce text.

Claude Computer Use (October 2024) demonstrated that an LLM could control a computer: look at screenshots, reason about what to do, move the mouse, click, and type. The model generated coordinate-based actions as tokens in its output space.

Google Gemini 2.0 (December 2024) introduced native **tool use**: the model could call Google Search, execute code, and query APIs as part of its generation process. These were not post-hoc integrations but core capabilities of the model.

LLaMA 3.1 (July 2024) included **function calling** as a first-class capability, enabling structured API interactions.

Manus and **OpenManus** (2025) built agentic frameworks where LLMs could plan, execute, and revise multi-step tasks autonomously—researching topics, writing reports, deploying code, and interacting with external services through defined tool interfaces.

The language model was evolving from a text generator into an **agent**: an entity that perceives, reasons, plans, and acts.

6.6 The Mamba in the Room

As 2025 progressed, the architectural landscape remained unsettled. Transformers dominated, but alternatives were gaining ground:

Mamba-2 had proven that SSMs could match transformers at scale, with linear complexity. **RWKV-7** (Goose, 2025) introduced context-dependent gating and dynamic state expansion, continuing the trend of rapid SSM improvement. **Jamba** showed that hybrid architectures could combine the strengths of both paradigms. **BitNet b1.58** (2024) demonstrated that models with only ternary weights $(-1, 0, 1)$ could match FP16 quality at a fraction of the memory—a path toward truly efficient on-device AI.

The question was no longer "can we build a better architecture than the transformer?" It was "which architectural innovations will be adopted by the next generation of frontier models?"

6.7 The Unresolved Questions

As this story draws to a close, the field faces profound unresolved questions:

The Alignment Problem: Can we ensure that increasingly capable AI systems remain aligned with human values? Constitutional AI, RLHF, DPO, and interpretability research have made progress, but the core challenge remains unsolved. As models gain agency and reasoning capability, the stakes grow higher.

The Data Wall: High-quality training data is finite. We have already trained on most of the public internet. Where will the next generation's training data come from? Synthetic data? Embodied experience? Better data efficiency?

The Economics: Training frontier models costs hundreds of millions of dollars. Inference at scale costs even more. Can the economics of AI be made sustainable, or will we see consolidation into a handful of well-funded labs?

The Open Question: Will the future of AI be open or closed? The open-source movement has demonstrated remarkable capability, but closed labs retain advantages in scale and coordination. DeepSeek proved that efficient architectures can challenge scale advantages, but will the next breakthrough favor openness or concentration?

The Architecture Question: Will transformers remain dominant, or will SSMs, hybrid architectures, or something entirely new displace them? The history of AI is a history of dominant paradigms being overturned—and each overturning begins before anyone recognizes it.

The Intelligence Question: What does it mean for a machine to reason? To understand? To be intelligent? The o1 and R1 papers suggest that reasoning can emerge from optimization, that chains of thought are a learnable behavior rather than a programmed one. But is this **real** reasoning, or a sophisticated imitation? The question is philosophical as much as technical, and it will define the next chapter of this story.

The story of AI began in 1943 with a mathematical proof that thoughts could be mechanized. Eighty years later, we have machines that can reason, create, and act in the world. The dream of McCulloch and Pitts—that the mind is a machine, and that building a mind means building the right machine—has been brought closer to reality than anyone in 1943 could have imagined.

But the story is far from over. The next chapter is being written right now.

Epilogue: What Comes Next?

As we close this story, the frontier continues racing forward. Models surpass PhD students on specialized exams. They control computers. They generate and execute code. They reason through problems that require dozens of steps. And yet, we have not reached the end of this narrative—far from it.

The next chapters are being written right now. Researchers are exploring architectures that go beyond attention entirely. New training paradigms promise to be 100× more efficient. Models are learning to verify their own outputs, to plan over long horizons, to interact with the physical world through robotics. The line between language models and general intelligence grows thinner each year.

But if this history teaches us anything, it is this: progress comes not from single breakthroughs, but from the patient accumulation of ideas—ideas that build on each other, correct each other, and compound into capabilities that no one could have predicted from the start. The transformer paper was rejected from ICLR before it changed the world. GPT-2 was deemed too dangerous to release. BERT was just a better pre-training recipe. And yet each of these, in combination with every other advance catalogued in these pages, built the world we now inhabit.

The story of AI is the story of human ingenuity applied to the hardest problem we've ever faced: understanding intelligence itself. And we are still in the early chapters.

— *The Story of AI, May 2026*